EC8206 — Functional Programming

Project Report

Adeesha M.G.P | EG/2021/4385

1. Expression interpreter

The program evaluates arithmetic expressions containing numbers, variables, addition, subtraction, multiplication, and division. It also supports local bindings with `Let` and conditional expressions with `If`.

`Expr.hs` contains the evaluator and simplification functions. `Main.hs` runs sample expressions, while `Tests.hs` checks evaluation, simplification, and batch processing. `comparison.py` implements the same evaluation rules in Python.

Run these commands from the extracted submission folder:

```
runghc -iProject Project/Main.hs
runghc -iProject Project/Tests.hs
python3 -B Project/comparison.py --parity
```

The program uses the following types:

```
data Expr = Lit Double | Var String
  | Add Expr Expr | Sub Expr Expr | Mul Expr Expr | Div Expr Expr
  | Let String Expr Expr | If BExpr Expr Expr
data BExpr = BLit Bool | Less Expr Expr | Equal Expr Expr
  | And BExpr BExpr | Or BExpr BExpr | Not BExpr
type Env = [(String, Double)]
eval :: Env -> Expr -> Either String Double
```

`Right` contains a successful result. `Left` contains an error message. Variable lookup uses the first matching name in the environment. A division checks the calculated denominator, so zero is detected even when it comes from a larger expression. NaN, infinity, and arithmetic overflow are also reported as errors.

Sample results

The environment is `[("x", 5), ("y", 2)]`. Each result below is checked against its expected value in `Main.hs`.

Expression	Expected	Actual
2 + 3 * 4	14	14
(x - 1) / y	2	2
10 / (x - 5)	Division by zero	Division by zero
missing + 1	Undefined variable: missing	Undefined variable: missing
let a = 3 in a*a + x	14	14
let x = 10 in x + y	12	12
(let x = 10 in x) + x	15	15
if true then 7 else 1/0	7	7
if false && 1/0 < 2 then 1 else 9	9	9
let unused = 1/0 in 7	Division by zero	Division by zero

The shadowing example changes x only inside the binding. The next example shows that the outer x remains 5. The conditional examples show that a skipped branch is not evaluated. However, the definition of a `Let` is evaluated even when its body does not use that variable.

2. Higher-order functions and simplification

Batch evaluation uses partial application:

```
evalAll :: Env -> [Expr] -> [Either String Double]
evalAll env = map (eval env)
```

After receiving `env`, `eval env` is a function waiting for an expression. `map` applies that function to every expression in the batch. `evalBatch` uses `foldr` to collect successful values, and `successfulExpressions` uses `filter` to select expressions that evaluate successfully. Their original order is kept.

For the ten samples, batch processing gives seven successful values: `[14, 2, 14, 12, 15, 7, 9]`. The other three results are two division errors and one undefined-variable error. The summary retains those messages alongside the success and failure counts.

Simplification rules

The tree is simplified from the children upward. For example, `(2 + 3) * 1` first becomes `5 * 1`, then becomes `5`. The `rewrite` function receives the arithmetic and Boolean rewrite functions as arguments, so the traversal can be reused with different rules.

`simplify` implements algebraic rules such as `x + 0 → x`, `x * 1 → x`, and `x * 0 → 0`. These rules need care when an expression can fail. For example, `missing * 0` produces an undefined-variable error, but replacing the whole expression with zero removes that error.

The program therefore also provides `simplifySafe`. It folds expressions that can be evaluated successfully without external bindings and keeps terms that might fail. For an unbound variable:

```
let e = Mul (Var "missing") (Lit 0)
eval [] e                 -- Left "Undefined variable: missing"
eval [] (simplify e)      -- Right 0.0
eval [] (simplifySafe e)  -- Left "Undefined variable: missing"
```

The distinction is also important for Boolean expressions. `false && error` does not evaluate the right operand, but `error && false` still fails because the left operand is evaluated first. The safe simplifier keeps that behavior.

Testing

The project compiled with GHC 9.6.6 using `-Wall -Wincomplete-patterns -Werror`. All 45 interpreter checks and ten sample checks passed. The tests cover operand order, missing variables, zero divisors, local scope, skipped branches, non-finite values, and ordered batch results.

A further 500 generated trees were checked in three environments, giving 1,500 comparisons between evaluation before and after safe simplification. Of these, 912 evaluated to errors. The same trees were checked for repeated simplification producing the same tree and for simplification not increasing tree size.

The Python tests passed, and the Haskell and Python implementations agreed on 16 comparison cases. Temporary changes to the addition evaluator and safe simplifier caused test failures, confirming that the tests detect these mistakes.

3. Comparison with Python

The Python evaluator uses tagged tuples and an `if`/`elif` sequence to select an operation. It raises `EvaluationError` for expected evaluation failures. The batch function catches those exceptions and appends each result to a list.

In Haskell, errors are represented by `Either String Double`. In a `do` block, a `Left` result stops the calculation and passes the error upward. In Python, an exception propagates through the recursive calls until it is caught. Both implementations handle errors, but Haskell shows the possibility of failure in the return type.

For local bindings, Python copies the environment dictionary before inserting the new value. This prevents the local binding from changing the caller's variables. Haskell creates an extended environment with `(name, value) : env`. The original list is unchanged, and its tail can be shared safely.

The Haskell constructors restrict the number and types of operands. For example, `Add` needs two `Expr` values, and `If` needs a `BExpr` condition. An expression with a string operand or a numeric condition is rejected at compile time. Python's tuple representation does not provide those static checks. It detects unknown tags or invalid fields when the affected code runs.

Pattern coverage warnings help find missing constructors, but they do not prove that an implementation calculates the correct answer. An `Add` case using subtraction would still compile, which is why numerical tests are needed.

4. Functional programming principles

Pure functions. Evaluation and simplification use only their arguments and perform no input or output. The same expression and environment give the same result. This allows a test to compare a returned value directly with an expected result. Printing is handled separately in the entry-point files.

Immutability. A local binding creates a new environment rather than changing an existing one. Simplification also returns a tree without modifying the original. The original and simplified expressions can therefore be evaluated and compared independently.

Type safety. The expression types prevent malformed tree structures, while `Either` separates errors from numbers. Runtime checks are still necessary for zero divisors, missing variables, and overflow. The types help organize those checks; they do not remove the need for them.

Limitations. The program evaluates trees built in Haskell; it does not parse text input. `Double` arithmetic still has rounding and underflow. The algebraic simplifier can remove errors, so the safe version is used for preservation tests. The safe pass may repeat work while evaluating subtrees. The generated tests cover a bounded set of inputs rather than every possible expression.

References

- EC8206 lecture notes: Higher-Order Functions and Pattern Matching, University of Ruhuna.
- G. Hutton, Programming in Haskell, 2nd edition, Cambridge University Press, 2016.
- GHC User's Guide, warning options: https://downloads.haskell.org/ghc/9.6.6/docs/users_guide/using-warnings.html
- Python documentation, unittest: https://docs.python.org/3/library/unittest.html